

NutriGuide Backend — Explained from Scratch

Think of the **backend** as the “kitchen” of a restaurant. The frontend (React app) is the customer who places an order. The backend receives that order, processes it, talks to the database (the storage), and sends back the result.

What Does the Backend Do?

The NutriGuide backend does 4 main jobs: 1. **Receives requests** from the frontend (e.g., “give me today’s meals”) 2. **Checks if the user is allowed** to make that request (authentication) 3. **Talks to the database** to get or save data 4. **Sends a response** back to the frontend

Backend Folder Structure — What Each File Does

backend/

server.js	←	The MAIN DOOR - starts the server, receives all requests
config/		
db.js	←	DATABASE CONNECTION - connects Node.js to MySQL
middlewares/		
auth.js	←	SECURITY GUARD - checks if the user is logged in
routes/		
authRoutes.js	←	MENU for login/register/profile URLs
recipeRoutes.js	←	MENU for recipe URLs
mealRoutes.js	←	MENU for meal-log URLs
helpRoutes.js	←	MENU for help/FAQ URLs
controllers/		
authController.js	←	CHEF for auth - actual login/register logic
recipeController.js	←	CHEF for recipes - actual recipe logic
mealController.js	←	CHEF for meals - actual meal-log logic
helpController.js	←	CHEF for help - actual FAQ logic

Step 1: server.js — The Entry Point (The Main Door)

When you run the backend, `server.js` starts first.

Think of `server.js` as the **front desk of a hotel**. All guests (requests) enter through here, and the desk tells them which department to go to.

// server.js does these 3 things:

// 1. Starts listening on port 5000

```
app.listen(5000);
```

// 2. Allows the React frontend to talk to it (CORS)

```
app.use(cors(...));
```

// 3. Connects each URL group to the right route file

```
app.use('/api/auth', authRoutes); // anything starting with /api/auth goes to authRoutes
```

```
app.use('/api/recipes', recipeRoutes); // anything starting with /api/recipes goes to recipeRoutes
```

```
app.use('/api/meals', mealRoutes); // anything starting with /api/meals goes to mealRoutes
```

```
app.use('/api/help', helpRoutes); // anything starting with /api/help goes to helpRoutes
```

Step 2: config/db.js — The Database Connection

The backend needs to talk to **MySQL** (where all data is stored). `db.js` creates that connection once and shares it everywhere.

// It creates a "pool" of up to 10 connections to MySQL

```
const pool = mysql.createPool({
  host: 'localhost', // where MySQL is running
  user: 'cdac',       // MySQL username
  password: 'cdac',   // MySQL password
  database: 'nutriguide', // which database to use
  connectionLimit: 10 // up to 10 simultaneous queries
});
```

A **connection pool** is like having 10 phone lines — so 10 requests can talk to the database at the same time without waiting.

The Database — 3 Tables

The MySQL database `nutriguide` has **3 tables** (like 3 Excel sheets):

Table 1: users

Stores every registered user. | Column | Type | What it stores | |—|—|—| | **id** | INT (auto) | Unique number for each user | | **name** | VARCHAR | “Moraya”

email	VARCHAR	"moraya@gmail.com" (must be unique)
password	VARCHAR	Hashed password (never plain text!)
role	ENUM	'user' or 'admin'
age, weight, height	numbers	Health data
diet_preference	ENUM	'vegan', 'vegetarian', 'non-vegetarian', 'none'
health_goal	ENUM	'lose_weight', 'maintain', 'gain_muscle'

Table 2: recipes

id	INT (auto)	Unique recipe ID
title	VARCHAR	"Dal Tadka"
ingredients	JSON	["dal", "ghee", "turmeric"]
calories	INT	320
protein, carbs, fat	INT	Nutrition values
diet_tag	VARCHAR	'vegan', 'vegetarian', 'non-vegetarian'
image_url	VARCHAR	URL of the dish image

Table 3: meal_logs

id	INT (auto)	Unique log ID
user_id	INT (FK)	Links to users.id
recipe_id	INT (FK)	Links to recipes.id
date	DATE	2026-05-09
meal_type	ENUM	'breakfast', 'lunch', 'dinner', 'snack'

FK (Foreign Key) means it's a reference to another table.
 user_id in meal_logs points to the actual user in the users table.

Step 3: Routes — The “Menu” (URL Map)

Routes are like a **menu in a restaurant**. They map each URL to the correct function.

Auth Routes (/api/auth/...)

POST	/api/auth/register	→ authController.register	(create account)
POST	/api/auth/login	→ authController.login	(log in)
GET	/api/auth/profile	→ authController.getProfile	(get my info) [needs token]
PUT	/api/auth/profile	→ authController.updateProfile	(edit my info) [needs token]

Recipe Routes (/api/recipes/...)

GET	/api/recipes/	→ recipeController.getAllRecipes	(all recipes) [p
GET	/api/recipes/recommendations	→ recipeController.getRecommendations	(my recs) [user]
POST	/api/recipes/	→ recipeController.createRecipe	(add recipe) [a
DELETE	/api/recipes/:id	→ recipeController.deleteRecipe	(remove recipe)

Meal Routes (/api/meals/...)

POST	/api/meals/	→ mealController.logMeal	(log a meal today) [user]
GET	/api/meals/daily	→ mealController.getDailyMeals	(see today's meals) [user]
GET	/api/meals/analytics	→ mealController.getAnalytics	(see weekly chart data) [user]
DELETE	/api/meals/daily	→ mealController.resetTodayMeals	(clear today's logs) [user]

Step 4: middlewares/auth.js — The Security Guard

A **middleware** is code that runs **in between** receiving a request and executing the controller. Think of it as a **security check at the airport** — everyone must pass through before boarding.

verifyToken — “Are you logged in?”

Request comes in with Authorization: "Bearer eyJhbGc..."

↓

verifyToken reads the token from the header

↓

jwt.verify(token, SECRET_KEY) - decodes and validates it

↓

If VALID → Attaches userId and userRole to the request → passes to controller

If INVALID → Returns 401 "Unauthorized" immediately

isAdmin — “Are you an admin?”

Only runs AFTER verifyToken passes

↓

Checks req.userRole === 'admin'

↓

If YES → Continues to controller

If NO → Returns 403 "Forbidden"

Step 5: Controllers — The “Chefs” (Business Logic)

Controllers contain the **actual logic** — they talk to the database and build the response. Each controller function: 1. Reads data from the request (**req.body**, **req.params**, **req.query**) 2. Runs a SQL query on the database 3. Sends a JSON response back (**res.json(...)**)

Example: authController.register

1. User sends: { name, email, password, age, weight, ... }
2. Check if email already exists in DB → if yes, return error

3. Hash the password: `bcrypt.hash(password, 10)`
4. INSERT new user into ``users`` table
5. Return: `{ message: "User registered", userId: 5 }`

Example: `authController.login`

1. User sends: `{ email, password }`
2. Find user in DB by email → if not found, return 404
3. `bcrypt.compare(sentPassword, storedHashedPassword)`
4. If match → create JWT token: `jwt.sign({ id, role }, SECRET, { expiresIn: '24h' })`
5. Return: `{ token: "eyJ...", user: { id, name, email, role, ... } }`

Example: `recipeController.getRecommendations` (Smart Logic)

1. Get user's `diet_preference`, `allergies`, `health_goal`, `weight`, `target_weight` from DB
2. Filter recipes by `diet_preference`:
 - `vegan` → only vegan recipes
 - `vegetarian` → veg + tiny % of vegan
 - `non-veg` → mostly veg + ~30% non-veg
3. Remove recipes that contain the user's allergies
4. Sort by calorie goal:
 - `lose_weight` → sort low calories first (500 cal)
 - `gain_muscle` → sort high calories first (600 cal)
 - `maintain` → random shuffle
5. Return top 10 recipes

How JWT (Token) Authentication Works — Full Picture

	FRONTEND -----	BACKEND -----
[Register]	POST <code>/api/auth/register</code>	Hash password, save to DB <code>{ message: "Success" }</code>
[Login]	POST <code>/api/auth/login</code>	Verify password with <code>bcrypt</code> <code>{ token: "eyJ...", user: {...} }</code>
	Save token to <code>localStorage</code>	
[Any page]	GET <code>/api/recipes/recommendations</code> Header: <code>"Bearer eyJ..."</code>	<code>verifyToken</code> middleware reads + verifies token attaches <code>userId</code> to req <code>recipeController</code> runs <code>{ recipes: [...] }</code>

JWT Token is like a **hotel key card**. The hotel (backend) gives it to you when you check in (login). Every time you want to open a door (protected route), you swipe the card. The door checks if the card is valid — it doesn't need to call the front desk every time.

Full Request Journey — Example: “User logs a meal”

1. USER clicks "Log Meal" on the Meals page
Frontend calls: POST `http://localhost:5000/api/meals/`
Body: `{ recipe_id: 12, date: "2026-05-09", meal_type: "lunch" }`
Header: Authorization: "Bearer eyJ..."
 2. `server.js` receives the request
Sees URL starts with `/api/meals` → sends to `mealRoutes.js`
 3. `mealRoutes.js` reads: POST /
First runs: `verifyToken` middleware
Reads the Bearer token, verifies it, sets `req.userId = 7`
 4. `verifyToken` passes → `mealController.logMeal()` runs
Reads `req.body`: `{ recipe_id: 12, date: "2026-05-09", meal_type: "lunch" }`
Reads `req.userId`: 7 (from the token)
Runs SQL:
`INSERT INTO meal_logs (user_id, recipe_id, date, meal_type)`
`VALUES (7, 12, "2026-05-09", "lunch")`
 5. MySQL saves the record and returns the new log ID
 6. Controller sends response back to frontend:
`{ message: "Meal logged successfully", logId: 45 }`
 7. Frontend receives the response and updates the UI
-

npm Packages Used (Dependencies)

Package	What it does
<code>express</code>	The web framework — handles routes and HTTP requests
<code>mysql2</code>	Connects Node.js to MySQL database
<code>bcryptjs</code>	Hashes passwords so they're never stored as plain text

Package	What it does
<code>jsonwebtoken</code>	Creates and verifies JWT tokens for authentication
<code>cors</code>	Allows the React frontend (port 3000) to talk to Express backend (port 5000)
<code>dotenv</code>	Reads secret values from the <code>.env</code> file (like DB password)

The `.env` File — Secret Configuration

The `.env` file stores **sensitive values** that you don't want to hard-code:

```
DB_HOST=localhost
DB_USER=cdac
DB_PASSWORD=cdac
DB_NAME=nutriguide
JWT_SECRET=your_secret_key
PORT=5000
```

These are read by `dotenv` at startup and used throughout the app via `process.env.DB_HOST`, etc.

Summary — How Everything Connects



middleware	middleware	middleware
authController (business logic)	recipeController (business logic)	mealController (business logic)

SQL Query

MySQL Database
users | recipes | meal_logs

Result

JSON Response sent
back to React